

Assignment 13.

1. Create a React component called PlaylistEditor that connects to a Firestore collection named 'playlists' and allows users to update the name of a playlist document by its ID.
2. Build a Delete button in your PlaylistEditor component that removes a playlist document from Firestore when clicked, and confirm deletion with a window.confirm popup before proceeding.
3. Implement real-time updates in a React component called LiveCommentsFeed that listens to a Firestore collection 'comments' using onSnapshot(), and displays the latest comments for a song without needing to refresh the page.
4. Explain in your own words (3-4 sentences) how Firestore's onSnapshot() keeps your app UI in sync with the database automatically, using your LiveCommentsFeed as an example.
5. Use ChatGPT or Copilot to help you write a function in your LiveCommentsFeed component that unsubscribes from the Firestore onSnapshot() listener when the component unmounts.

 Hint: Ask the AI tool for the correct cleanup pattern in useEffect with Firestore listeners.

Answer:- 1 & 2. PlaylistEditor Component (Update + Delete Playlist)

```
import { useState } from "react";
import { db } from "../firebase";

import {
  doc,
  updateDoc,
  deleteDoc
} from "firebase/firestore";

const PlaylistEditor = () => {
  const [playlistId, setPlaylistId] = useState("");
  const [playlistName, setPlaylistName] = useState("");

  // Update Playlist
  const handleUpdate = async () => {
    if (!playlistId || !playlistName) {
      alert("Please fill all fields");
      return;
    }

    try {
      const playlistRef = doc(
        db,
        "playlists",
        playlistId
      );
    }
  }
}
```

```

    await updateDoc(playlistRef, {
      name: playlistName,
    });

    alert("Playlist Updated Successfully");

    setPlaylistId("");
    setPlaylistName("");
  } catch (error) {
    console.log(error);
  }
};

// Delete Playlist
const handleDelete = async () => {
  const confirmDelete = window.confirm(
    "Are you sure you want to delete this playlist?"
  );

  if (!confirmDelete) return;

  try {
    await deleteDoc(
      doc(db, "playlists", playlistId)
    );

    alert("Playlist Deleted Successfully");

    setPlaylistId("");
    setPlaylistName("");
  } catch (error) {
    console.log(error);
  }
};

return (
  <div>
    <h2>Playlist Editor</h2>

    <input
      type="text"
      placeholder="Playlist ID"
      value={playlistId}
      onChange={(e) =>
        setPlaylistId(e.target.value)
      }
    />
  </div>
);

```

```

    <br />
    <br />

    <input
      type="text"
      placeholder="New Playlist Name"
      value={playlistName}
      onChange={(e) =>
        setPlaylistName(e.target.value)
      }
    />

    <br />
    <br />

    <button onClick={handleUpdate}>
      Update Playlist
    </button>

    <button
      onClick={handleDelete}
      style={{ marginLeft: "10px" }}
    >
      Delete Playlist
    </button>
  </div>
);
};

export default PlaylistEditor;

```

3. LiveCommentsFeed Component (Real-Time Firestore Updates)

```

import { useEffect, useState } from "react";
import { db } from "../firebase";

import {
  collection,
  onSnapshot
} from "firebase/firestore";

const LiveCommentsFeed = () => {
  const [comments, setComments] = useState([]);

  useEffect(() => {
    const commentsRef = collection(
      db,

```

```

      "comments"
    );

    const unsubscribe = onSnapshot(
      commentsRef,
      (snapshot) => {
        const commentsData = snapshot.docs.map(
          (doc) => ({
            id: doc.id,
            ...doc.data(),
          })
        );

        setComments(commentsData);
      }
    );

    return () => {
      unsubscribe();
    };
  }, []);

  return (
    <div>
      <h2>Live Comments Feed</h2>

      {comments.map((comment) => (
        <div key={comment.id}>
          <p>{comment.text}</p>
        </div>
      ))}
    </div>
  );
};

export default LiveCommentsFeed;

```

4. Explanation of onSnapshot()

onSnapshot() creates a real-time listener between Firestore and your React application. Whenever a document is added, updated, or deleted in the comments collection, Firestore automatically sends the latest data to the listener. In the LiveCommentsFeed component, the callback function updates the React state with the newest comments, causing the UI to re-render instantly. Because of this, users see changes immediately without refreshing the page.

5. Cleanup Function for Firestore Listener

```
useEffect(() => {  
  const unsubscribe = onSnapshot(  
    collection(db, "comments"),  
    (snapshot) => {  
      // Handle data  
    }  
  );  
  
  return () => {  
    unsubscribe();  
  };  
}, []);
```